Your Data, Your Rules: Extending the Decentralized Web

Jacob Hoffman
web10 Inc.
jacob@web10.app

ABSTRACT

Decentralized platforms distribute data across providers, but access control often remains tied to each platform's identity system and application model. We present **web10**, a user-addressed data platform that represents content and access relationships in a shared columnar store. Applications query service-namespaced documents through a uniform permission model in which principals from different nodes can hold roles in user-managed groups. The node enforces access at the query boundary by supplying reader-filtered relations before joins and aggregations execute. Clients communicate directly with the relevant nodes rather than relying on server-to-server content federation. We describe the architecture and demonstrate cross-application reuse: a game and a marketplace read social-written profiles without modifying the social application or adding application-specific node endpoints. Connected browser flows exercise consent and revocation. An isolated two-node experiment exercises foreign-principal reads and local grant revocation while reusing cached home-node certification.

1 INTRODUCTION

Email is the decentralized web's proof of concept. A global identifier (`you@provider`), a sovereign store per provider, and standardized server-to-server mail transport let you send a message across providers—no central authority or shared store. Each provider's server is independent; the protocol and the identifier are what make the network work.

Email decentralized the *data plane* for one format: mail. It did not decentralize *access*. Your contacts, your blocking, your sharing permissions live inside one provider's identity system. You can email a Yahoo user from Gmail, but you cannot grant that Yahoo user *editor* on your Google Calendar. The payload crossed the boundary; the permissions did not. Every social platform inherits this split: the content can be portable in principle, but access relationships remain locked to the platform's backend: who follows whom, who belongs to a group, and who may read or contribute.

web10 extends the email model in two directions at once. First, it generalizes the data plane from one format to *all* formats: a shared columnar store in which any application defines its own services and queries, so a post, a profile, a game score, and a marketplace listing are all documents in the same store, addressed by the same primitives. Second, and more consequentially, it decentralizes the *access layer* itself: a uniform, user-managed permission vocabulary in which a grant is a row naming a *globally addressable principal* (`provider/username`), a group, a service, and an operation. Any principal on any node can hold any role in a group. The user is the policy administrator for their own data, and that policy reaches across the network.

Table 1: Capability comparison. Email is the model web10 generalizes, not a social system. Social platforms decentralize or scale the data plane but keep access control platform-defined and coarse-grained; Solid decentralizes data *and* some access, but its model is resource-centric. web10 is the only system with all five.

Capability	Email	Mastodon	Bluesky	Solid	web10
User-owned data, portable	~ mail	~	✓	✓	✓ any
Fine-grained, user-mgmt access	×	×	×	~ res.	✓ role
Access cross-node-addr.	~ addr	×	×	~ Pod	✓
Composable, app-def. query	×	×	~ view	~ dep.	✓ SQL
User-addr. net. (no fed.)	✓	×	×	✓	✓

The result is a system that combines properties prior work treats as a tradeoff. Table 1 places the deployed systems and the email model that web10 generalizes against five capabilities. The marks are ✓ (present), × (absent), or ~ (partial / scoped).

The pattern is the email split, repeated. Email decentralized where data *goes*—a global address, sovereign stores, inter-provider mail transport—but not who may *access* it: you can mail a Yahoo user from Gmail, but you cannot grant that user *editor* on your Calendar. The social platforms inherit the same split at larger scale. Mastodon [11] and Bluesky [10] let you follow someone on another server and carry your data in principle, but their authorization is platform-defined and coarse-grained—follow/block and collection-level visibility, not per-principal grants over application-defined services. Facebook scales the data plane to a massive lakehouse but keeps the access model entirely inside its own platform. Solid [15] is the closest prior work: it decentralizes data *and* some access, with WAC group agents [5] and ACP client/issuer matchers [3]. But its access model is *resource-centric*—ACLs on linked-data resources—rather than a uniform role/grant vocabulary over services that applications define, and its cross-Pod query cost depends on deployment. web10 claims the center: all five capabilities at once, over a network that is user-addressed rather than server-federated.

The platform is deployed. web10-social is a creator-oriented social application —feeds, groups, direct messages, discovery, short- and long-form video, music—built entirely on the generic node: its persistent data and authentication use web10's document, group, query, and media interfaces, with no separate social-data backend. The node also serves a game and a marketplace that reuse the social application's profile documents through the same access boundary, without a node change or an application-specific endpoint.

This paper makes three contributions:

(1) **A user-addressed network model** (§3): a federation of sovereign nodes in which the user's client mediates cross-node reads, each node re-authorizes every read against a verified token, and the access model is globally addressable by construction—any principal on any node can hold any role in a group. The model generalizes email's architecture from one data format to all formats and from content routing to access control.
(2) **A reader-specific query boundary** (§4): application-written SQL over a shared columnar store, in which service references resolve to reader-filtered CTEs *before* the caller's joins and aggregation. The access decision is made on the input, not patched onto the output. The implementation combines policy resolution, SQL validation, and boundary construction above a shared database account.
(3) **A deployed platform and connected composition** (§5, §6.1): a social application, a game, and a marketplace over the same node, with real browser consent, cross-author joins, and revocation exercised. We report the metadata fan-out cost of the access layer and the batching decision that addresses it, and exercise home-node-certified reads across two isolated nodes (§6.2).

The evaluation in §6 reports the connected interoperability checks and the metadata-resolution measurement. §7 relates the design to decentralized storage, IAM, and multi-tenant databases; §8 concludes.

2 THE ACCESS MODEL

2.1 Two trust layers

For a group-scoped content operation, the model distinguishes an *app*, a *principal* (the acting user), a *group* G, a *service* S (a data namespace such as `posts` or `comments`), and an *operation* O (e.g. `readAll`, `create`). Authorization asks two questions where the corresponding gates apply:

Layer	Question
App trust (outer wall)	Can this *app* perform O on S?
Person trust (inner wall)	Does membership or a class/role grant allow O on S in G?

The app contract expresses trust in a calling application. In the current document and query paths, authenticated browser-style tokens are checked against an origin-keyed contract when an HTTP `Origin` is present; without that header, this app-contract check is skipped. Anonymous queries also skip it. OAuth-delegated tokens instead select a `client_id`-keyed contract, including when no Origin is present. The node enables wildcard, non-credentialed CORS; endpoint code enforces application grants separately. Origin identifies the browser calling context, while the delegated token carries the OAuth client identity. Non-browser clients control their own Origin headers. Identity verification and document/group authorization are separate checks.

The group layer expresses trust in people. The split lets a user revoke an app grant without leaving a group, or leave a group without changing app grants.

2.2 Roles are per-service permission maps

A role is a named bundle of per-service permissions—a map from service to the operations it grants:

```
{ "name": "editor",
  "permissions": {
    "posts":    ["readAll", "create", "updateOwn"],
    "comments": ["readAll"]
  } }
```

The map *is* the scope. `'*'` wildcards all document services; a reserved key `'group'` carries *structural* operations on the group itself (`manageRoles`, `assignRoles`, `deleteGroup`, join/member operations), while *content* operations—including moderation such as `hideAll`—live under the service key, because they act on documents, not on the group's shape.

Two properties keep the model small. First, the role map is the *same shape* as the app-contract permission object: there is one permission language across both trust layers. Second, a person holds *one explicit member role* per group; compound permissions can be placed in that role rather than stacked through multiple assignments. Policies use positive grants; explicit deny and context-dependent conditions are outside this model. A group defines whatever roles fit its purpose (`owner`, `moderator`, `page-curator`, `member`, or `dj`, `listener`)— and applications supply their role definitions. Some lifecycle operations use conventional names such as `member`.

2.3 Principal classes and effective permissions

Access is granted to *principals*, of which there are three nested classes, broadest to narrowest:

Class	Who
`anyone`	every request, signed in or not
`authenticated`	a verified non-anonymous user, member or not
`member`	holds a member row in this group

The broad classes `anyone` and `authenticated` are represented by *reserved keys in the member table*; individual members use their own principal keys. No separate visibility table is needed. For example:

```
group_members:
  (G, 'anyone',        'reader')    -- the group is
       public
  (G, 'authenticated', 'reader')    -- signed-in users
       can read
  (G, 'bob',           'moderator') -- bob moderates
  (G, 'alice',         'owner')     -- alice owns
```

A principal's *effective* permissions in a group are the union of the permission maps of every class they belong to:

$$\begin{aligned}\text{effective}(P, G) \;=\; &\text{grant}(G,\ \texttt{anyone})\\ \cup\ &\big(P \in \texttt{authenticated}\ ?\ \text{grant}(G,\ \texttt{authenticated}) : \emptyset\big)\\ \cup\ &\big(P \in \texttt{member}(G)\ ?\ \text{grant}(G,\ \text{role}(P)) : \emptyset\big)\end{aligned}$$

The content gate combines this grant union with membership admission. In `can_read_group` and `can_write_group` helpers, an active member is admitted for content reads and creates without requiring service-specific `readAll` or `create` in that member's role. For non-members, the class grants supply those permissions. Other role-controlled operations use their respective permission checks.

The union is monotone across principal classes under otherwise equal conditions. Blocks, sharing state, hidden documents, and node moderation apply additional visibility restrictions to the resulting content.

2.4 Publicness is a grant, not a flag

There is no `visibility` column. "Make my group public" is the act of granting `readAll` to `anyone`; "signed-in only" grants it to `authenticated`; "private" withholds those non-member grants. Join policy is orthogonal: it controls how a human *becomes* a member (open, request, invite-only) and says nothing about what a non-member bystander can see. A group can be invite-only to join and still publicly readable—the discovery board is exactly that.

This is the sense in which web10 brings IAM to the front: the operations a cloud operator would express as policies—*principal class, service, operation, grant*—are the operations an *end user* expresses about their own data. A grant is represented by metadata rows and revoked through tombstones. Subsequent checks use latest-row resolution. End-to-end revocation latency comprises database visibility, concurrent checks, in-flight requests, and the remaining lifetime of issued capabilities.

3 A USER-ADDRESSED NETWORK

The access model of §2 is defined for one node. This section states the network model that makes it a federation rather than a single deployment, and the property that distinguishes web10 from both email and from server-federated systems like ActivityPub.

3.1 Email's architecture, generalized

Email combines globally addressed accounts (`you@provider`), sovereign provider stores, standardized inter-provider mail transport, and no central shared mailbox database. Its servers do exchange mail with other servers; client-mediated cross-node reads are web10's transport choice, not a property inherited from email.

web10 retains independent providers and global addressing. The identifier is `provider/username`—the provider is the node, the username is the user, and the pair is globally unique across the federation. The store is a shared columnar data plane (one `documents` table per node, keyed by `(author_key, doc_id)`), not a per-user mailbox, but it is still sovereign: each node's store belongs to that node, and a user's home node is where their account and their grants live.

The data and access extensions are: (1) the data is no longer locked to one format—any application defines its own services over the same store, so the network carries posts, profiles, scores, listings, and whatever an app names; and (2) the *access layer* is decentralized, not just the data layer. A grant is a row `(group, principal, role)` in which the principal is the global `provider/username`. A user on node A can be granted `editor` on a group that lives on node B. The permission vocabulary of §2 —principal class, service, operation, grant—is the same whether the principal is local or three nodes away. Email let your data leave the building; web10 lets your *permissions* leave it too.

3.2 The linchpin: verified, not registered

A user's home node is where their account lives. When that user reads content on another node, they do not sign up there. Their client presents their home-node token; the foreign node obtains home-issuer certification, resolves the principal to `provider/username`, and applies its own group-membership and grant checks. The user's identity is *verified, not registered*, on the foreign node. Consequences:

- A user's following set can span N nodes; they hold exactly one account, on their home node.
- A node's data is read only by holders of a verified token that the node's own membership and grant checks authorize—whether the token was minted by this node or a foreign one.
- "Following someone on another node" is a *cross-node membership*: a member row on the foreign node referencing a foreign principal. The content never crosses the boundary; only the membership reference does.

3.3 The boundary: the HTTP API and a verified token, never the database

A node never queries another node's store. The cross-node boundary is the *HTTP API* with a *verified token*. A foreign node's API call passes through that node's normal authentication and access gate, exactly like a local call. We reject database-level cross-server mechanisms (ClickHouse's `remote()`/`cluster()`) for federation: they are for a trusted cluster under one operator, they couple schemas across independent nodes, and they create a DB-level trust path that bypasses the API's authorization entirely—a node operator's database credentials would become a federation credential.

This yields an invariant that the whole model is built to hold:

I6. The cross-node boundary is the HTTP API and a verified token; remote data is content, never control. A foreign node's response is untrusted content—the same category as a post body. It is never query input, never a key into the local store, and never a grant.

I6 is what defeats the obvious attack. A malicious node B cannot "mine" node A during a cross-node read, because there is no cross-node *query*: A never reads B's store, and B's API responses reach A only as JSON content that A has already authorized for this reader. The feed merge is a result-set *union* of independently authorized reads; B's data are values being merged, never parameters to a query against A. A data reference is not a grant: if B's content points the client at an A-node document, A re-authorizes that fetch against the reader's token and denies it if the reader is not authorized. The same property that stops a local user reading other posts (I3) stops a foreign node from steering a read—federation does not weaken it, it adds the verification step so the serving node can check a foreign token before applying I3.

3.4 The read model: client-side fan-out

When a user's feed spans nodes, the *client* makes the cross-node calls. The client holds the user's home-node token, groups the user's following set by provider, and queries only the distinct nodes the user follows people on— targeted, not broadcast. Each node returns its own access-gated result set; the client performs a ranked merge. No node proxies or caches another node's data,

so there is no node-to-node credential to steal and no stale-cache confusion. The node remains a dumb broker (D60): it does not learn to "merge feeds."

3.5 Domain as portable alias: the address survives provider changes

The `provider/username` principal is the address, but the username need not be an opaque string. Through the authenticator's claim flow, a user can bind a DNS domain to their account: a TXT record at `web10.domain` carries `web10.v=1; provider=node;user=username`, and an authenticated `POST /v3/domains/verify` request makes the node perform the DNS lookup itself, checking that the record's provider matches the node and its username matches the authenticated caller before storing the verified alias. The record is a pointer, not a credential: DNS lookup is discovery, not authentication, and the username remains the principal— groups, tokens, and the read gate never key on the domain. The same pattern the web already uses for identity claims (DKIM, SPF, `OpenPGPkey`), applied to a social identity.

The practical consequence is portability in the strong sense: the domain is a real, resolvable, user-owned identifier that survives provider changes by construction. A user deplatformed by a mainstream platform can point their domain at a sovereign node that will not remove their content, and the followers who follow the domain resolve to the user's new location. The legal distinction is specific: follower relationships are the *user's* data, not the platform's. A platform that deletes a user's follower list is deleting the user's data—a different legal category than removing the platform's own content. web10 makes this concrete: the following relationships live in the user's sovereign data plane, addressed by the user's domain, and move with them to a node that will not delete them.

The enforcement path makes this concrete in the read model: web10 apps are implemented entirely in the UI, over the web10 API, with no platform backend in the read path. When the social client renders a user's feed, it pulls directly from the servers of the nodes the user is following—grouping the following set by provider and querying each node directly with the user's home-node token. The content arrives as authorized data, not as a platform-mediated feed.

This is the internet-freedom property that the email model proved and the social platforms lost: your address is yours, you can switch providers, and no single platform can unilaterally erase your inbox or block your senders. web10 restores that property for social data, and the domain-as-alias mechanism makes it concrete rather than conceptual.

3.6 What is running and what is the active build

The network model is the architecture; runtime maturity is phased. Foreign readers use `{provider}/users/{username}`. Local writers and readers retain legacy bare keys until a coordinated migration; canonicalization is not complete across the identity layer. For a foreign token, the serving node asks its home issuer's HTTPS `/certify` endpoint and caches a successful identity answer for that exact JWT until expiry. The issuer verifies with its own signing secret; the serving node still checks its own access grants on every read. This live-certification design does not require JWKS or shared signing secrets. The isolated experiment in §6.2 uses independently generated node keys and owner-managed foreign grants. Production key provisioning, foreign self-join, complete principal migration and client-side feed merging remain open. The current shared signing-key default must not be mistaken for independently provisioned production nodes.

4 ENFORCEMENT AT THE QUERY LAYER

4.1 One columnar table

A ranked feed has analytical components: joins across authors and engagement, filtering, aggregation, and sorting. A social application also has point reads, updates, and coordination-heavy operations. web10 chooses ClickHouse [17] for both structured content and operational metadata rather than maintaining a separate Postgres-to-ClickHouse replication pipeline. User content occupies one shared `documents` table keyed by `(author_key, doc_id)`. The user namespace is logical isolation within that table, not a per-user database or physical collection. Large media objects reside in object storage. The following simplified schema omits reference and advertising fields:

```
CREATE TABLE documents (
    doc_id          String,
    author_key      String,
    collection_name String,   -- the service: 'posts', '
        comments', ...
    body            String,   -- JSON
    tags            Array(String),
    created_at      DateTime64(3),
    updated_at      DateTime64(3),
    deleted         UInt8 DEFAULT 0
) ENGINE = ReplacingMergeTree(updated_at)
ORDER BY (author_key, doc_id);
```

Updates insert a newer version; deletes insert tombstones. Background merges alone do not establish read correctness: reads must select the latest version *including tombstones*, then filter deleted rows. Filtering tombstones before deduplication can resurrect the previous version. Documents attach to groups through `doc_groups`; `group_contracts` and `group_members` hold contracts and assignments. These are narrower metadata tables whose size grows with groups and memberships.

One structured store removes a cross-database synchronization dependency. Its operational costs include version accumulation, merge pressure, read-time deduplication, point-query latency, and shared-resource contention. Multi-row content/membership changes and concurrent revocation require explicit consistency handling. Backup and recovery span database metadata and object storage; these are requirements of the design.

4.2 The query boundary: a wall, not a membrane

Application-written SQL over a multi-tenant store requires a boundary before the caller's relational operations. The query API accepts a constrained read-only SQL surface, not arbitrary database access. Its intended boundary is a *wall, not a membrane*: validated service references read reader-filtered CTEs, rather than raw tables whose output is filtered afterward. Correctness

depends on validation coverage and SQL dialect interpretation, as discussed below.

Why output-side filtering is unsound. The natural design wraps the caller's query and filters the *result*, e.g. `SELECT * FROM (q) WHERE <filter>`. This fails because the caller's query has *already read the raw table* inside the parentheses, and an outer filter cannot undo an aggregation that baked everyone's data in:

```
-- counts every user's comments; the outer group filter
     is too late
SELECT author_key, count(*)
FROM documents WHERE collection_name = 'comments'
GROUP BY author_key
```

The aggregate has already collapsed all authors into counts before any row-level filter could run. Filtering the output of an aggregation is not access control.

Redirecting the input instead. web10 parses submitted SQL to an AST; it does not send that original string directly to ClickHouse. It validates table references and injects a *boundary CTE* under each referenced service name. The CTE deduplicates the service's documents, joins current `doc_groups` mappings, filters to readable groups, and applies block, sharing, hidden-document, and node-ban exclusions. Thus `SELECT ... FROM posts` resolves to a service CTE before aggregation or joins. The readable set can include public or authenticated-class grants, not just the reader's memberships. An empty set produces an empty service CTE. Before compilation, an authenticated request with explicit groups is rejected with 403 if none of its referenced services has a readable group. The query boundary does not automatically union in all reader-owned documents.

The walk and the backstop. The compilation is five steps: (1) parse to an AST (unparseable or multi-statement input is rejected); (2) check the statement kind (`SELECT` and supported set operations, not `INSERT`, `DROP`, etc.; this is not full SQL type checking); (3) walk every `Table` node, including inside caller CTEs and subqueries, rejecting raw node tables, ungranted services, and table functions (`file()`, `s3()`, `numbers()`, represented in the validated parser shapes as empty-named `Table` nodes); (4) inject the needed boundary CTEs ahead of the caller's CTEs and re-emit from the validated AST; and (5) *round-trip*—re-parse the compiled SQL and require it to come back as exactly one statement. The round-trip checks composition syntax. Targeted parser/engine tests exercise selected dialect and name-resolution cases. The executed query is emitted from the validated AST.

Why place policy above ClickHouse? web10 uses a shared database service account and constructs a reader-specific boundary combining membership, class grants, and visibility exclusions. This design puts policy orchestration in the application rather than provisioning a database account and policy for each end user. PostgreSQL policies offer another enforcement placement, using expressions, functions, and subqueries to consult dynamic authorization state [13]; an equivalent membership policy provides a comparison baseline for the cost of these enforcement placements.

Trusted implementation. The parser, validator, generated CTEs, identity resolution, metadata checks, and ClickHouse name-resolution semantics are in the trusted computing base. The service account must read raw tables to execute generated CTEs; its privileges do not independently enforce each reader's boundary. Repository tests cover selected top-level, caller-CTE, subquery, table-function, and aggregation cases, with a live-node E2E gauntlet for supported shapes. Parser and engine upgrades require renewed compatibility and adversarial testing.

4.3 Metadata resolution and relational cost

The read path is the heart of the design:

(1) **Resolve.** Take explicitly requested candidate groups, or discover the reader's memberships. Parse the query to identify its services.
(2) **Compute.** For each referenced service and candidate group, check active membership or resolve class grants through member and contract metadata. The current query endpoint calls `readable_groups`, which loops over candidates; a check may issue several metadata queries.
(3) **Execute.** Inject each service's readable-group set into its boundary CTE. ClickHouse performs document/mapping deduplication, the mapping join, visibility exclusions, and the caller's joins, aggregates, and sorting.

For s referenced services and g candidate groups, this path performs up to sg group-gate evaluations, with early membership success reducing work within a check. The repository also has a batched helper used elsewhere; the query endpoint currently uses the per-group helper. Role resolution does not need content bodies, but its metadata I/O and the generated relational work both contribute to authorization cost.

The document-to-group join can yield multiple rows for one document attached to several readable groups. Applications must choose whether to retain those associations or deduplicate before counting and pagination. CTE expansion, join cardinalities, exclusions, and sorting can change database work and query plans. Evaluation accounts for both metadata resolution and this relational work.

4.4 Isolation rule and regression coverage

The intended isolation rule is that another author's document requires an authorized group path, including public or authenticated-class grants (invariant I3). The query path implements this through service CTEs. Permission-matrix and exploit-regression tests exercise the compiler and endpoint boundaries. CRUD, directory, group-attachment, preparation, and media paths have separate test scopes; §6.5 records the current coverage and open boundaries.

4.5 Document-linked media capabilities

Structured documents refer to binary objects in MinIO/S3-compatible storage. The intended linkage is document authorization followed by capability issuance: presigned URLs let browsers fetch individual objects directly; HLS uses short-lived signed manifests and segment capabilities, with document checks on manifest access. Expiry limits capability lifetime but does not retract bytes already fetched or guarantee immediate revocation.

The query prepare pass combines media resolution, HLS signing, advertising, and avatar preparation. This keeps capability issuance next to result preparation while object storage serves the media bytes. Its authorization boundary includes key ownership, authorized cross-user references, result projections, public thumbnails, and stale capabilities (§6.5).

4.6 Data policy, not privacy

web10 deliberately trusts the node operator with readable content. Search, ranking, moderation, and audit functions operate on plaintext. Tokens and signed capabilities protect application/user boundaries; terms govern operator conduct. This trust model supports node-side querying without requiring default end-to-end encryption.

Here *user-owned* denotes logical authorship namespaces and user-controlled grants. The operator retains physical access to hosted data, and legal remedies depend on terms, jurisdiction, and enforceability. Usable export and exit are broader product goals. The isolated cross-node read slice is demonstrated in §6.2; interoperable provider migration and production federation remain forward scope (§3).

4.7 Implementation boundary

Python/FastAPI handles request validation, identity and permission orchestration, SQLGlot parsing/compilation, serialization, and result preparation. ClickHouse's native engine performs scans, joins, aggregation, filtering, and sorting. The node API source and TypeScript SDK are available alongside the query endpoint and boundary compiler. The production API command configures four Gunicorn processes with Uvicorn workers. This service/database division of work puts query execution in ClickHouse and orchestration in Python. Profiling separates API CPU, database CPU, I/O waits, and queueing to locate the costs of parsing, metadata round trips, response serialization, and preparation.

5 APPLICATION COMPOSITION

web10-social is the demonstration application built on web10: its persistent social data and authentication use web10 APIs and the SDK rather than a separate social-data backend. It is the platform's primary deployment, exercising the architecture through feeds, social groups, direct messages, and discovery. These features require different content shapes and audience relationships over the same document and group substrate. The feed is a useful lens because it combines ranking, relationships, media, and multiple authorization scopes. We examine its query composition here; workload cost is the subject of §6.

5.1 The killer-app hypothesis

web10-social is also the project's *killer app*: an application compelling enough to attract users to a new platform. The longer-term vision is an open, decentralized ecosystem in which many applications reuse a shared data and access substrate. Social is the initial adoption vehicle and a demanding exercise of that substrate; the sections that follow examine the breadth of what the deployed application actually runs.

Table 2: Media and interaction patterns covered by the demonstration. Platform names identify familiar interaction patterns.

Demonstration surface	Familiar reference
Communities, text posts and discussions	Facebook, Threads
Image and mixed-media posts	Instagram
Vertical short-video browsing	TikTok
Long-form video watch pages	YouTube
Audio-track browsing and playback	YouTube Music

The application spans a media spectrum from text and social relationships to images, short-form video, long-form video, and audio. Facebook, Instagram, Threads, TikTok, YouTube, and YouTube Music provide familiar reference points for these interaction patterns. Table 2 maps them to the application's implemented surfaces. These reference points describe interaction patterns; the point is that all of them compose on the same document, group, and query substrate.

Different media and interaction patterns share document storage, audience relationships, engagement records, and access machinery, while their presentation and queries are defined by the client. The architectural demonstration is that this range of social experiences can be built on web10's APIs. A successful adoption vehicle would bring those users and their relationships into the wider application ecosystem.

5.2 A frontend application over the shared node

The application is a React/TypeScript SPA. Its data client connects to the node; social operations express posts, conversations, relationships, and feeds through web10's document, group, query, and media interfaces. This is backendless from the application developer's perspective: web10 supplies the backend, while the application supplies its data conventions, queries, and interface. The browser build consists of static assets and can be hosted independently, for example in S3 behind HTTPS/CDN and SPA route handling.

The current deployment also includes a small Node link-preview renderer for crawlers. That presentation component uses web10 APIs and holds no separate social-data store; a static-only deployment needs a separate arrangement for crawler cards. Media bytes transfer through API-issued object-storage URLs, and real-time peer traffic uses the SDK's RTC support. Telemetry and external embeds are additional network paths. Application data and authentication use the node; media delivery and these ancillary connections follow their respective transport paths.

At source revision `317fa349e`, the application-local frontend contains **46,285 physical source lines across 141 files**: 45,846 TypeScript/TSX lines and 439 CSS lines under `src/`, excluding tests. Its local tests contain another 31,060 lines in 105 files; the link-preview renderer is 431 lines in three JavaScript modules. Counts include comments, blank lines, and local type declarations. They exclude public/generated assets, dependencies, configuration, tooling, and sibling shared-library and SDK implementations.

Implementation links for this application use that pinned revision; the later experiment snapshots are identified separately in §6.1 and §6.3.

5.3 The feed as a query

The application's feed implementation builds a query over `posts`, `reactions`, `comments`, and `profile`. It chooses chronological or weighted ranking, joins engagement aggregates and author profiles, and supplies cursor conditions. The node resolves those service names through the boundary CTEs. Preparation can attach advertising and resolve media, HLS manifests, and author avatars on returned rows. This separates the application's feed definition from a dedicated feed endpoint, while retaining explicit server-side preparation machinery.

Three implementation observations define what the feed path exercises:

(1) **Page assembly.** For a nonempty page with post IDs, `readFeedPage` issues the main query, then two engagement queries (reactions and comments) and a `contentViews` call. Group discovery precedes this work, and media bytes require separate fetches. UI-level reaction-state reads and other screen activity may add calls. Page assembly therefore combines one main composition query with scoped follow-up reads.

(2) **Authorization scope affects semantics.** The main query's engagement joins use the feed-group scope, while engagement documents may live in the discovery group. The follow-up queries use the union of feed groups and discovery to populate displayed counts. Counts are relative to readable relations, not necessarily global totals. The displayed counts can therefore differ from the counts used by the main ranking query.

(3) **The query is application-defined.** Ranking and filtering can change without adding a dedicated node feed endpoint. Applications can select their ranking and projections while reusing the same boundary construction and preparation services.

Evaluation will measure both the complete page-assembly path and its main query, recording client-call counts, internal metadata queries, and database work separately alongside latency and throughput.

5.4 Multiple applications, one data plane

The social application is a producer of reusable data, not the only client of the node. Two small static applications consume its `profile` documents: Snake associates completed runs with player identities; Marketplace associates listings with seller identities. Each writes its own service and reads the existing profile format (Table 3). Neither modifies web10-social or adds a game, commerce, leaderboard, or seller-profile endpoint to the node. The common dependency is the document convention and access primitives, not an application-specific API agreement.

The following abbreviated leaderboard query illustrates the seam. Both service names resolve to reader-filtered relations before the join. The full application also selects each player's best score and supplies profile-media preparation.

```
WITH latest_profiles AS (
  SELECT author_key, body FROM profile
  QUALIFY row_number() OVER (
    PARTITION BY author_key
    ORDER BY updated_at DESC, created_at DESC,
             doc_id DESC) = 1
)
SELECT s.author_key,
       JSONExtractInt(s.body, 'score') AS score,
       p.body AS profile_body
```

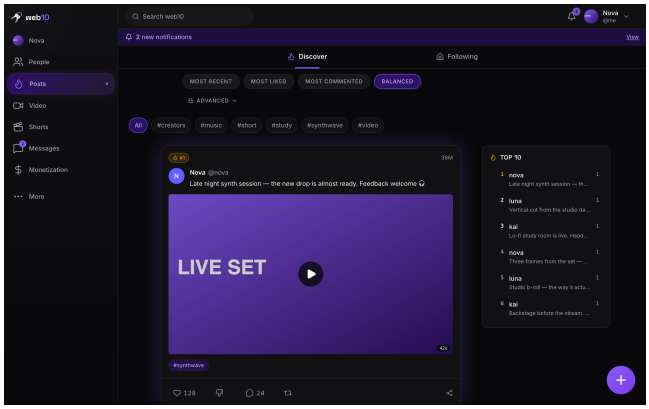


Figure 1: The web10 feed. The reader chooses the ranking (Most Recent / Most Liked / Most Commented / Balanced) and filters (tags); each is a different application query over the same columnar store. The main query uses reader-specific service boundaries; engagement and view metrics also use follow-up calls.

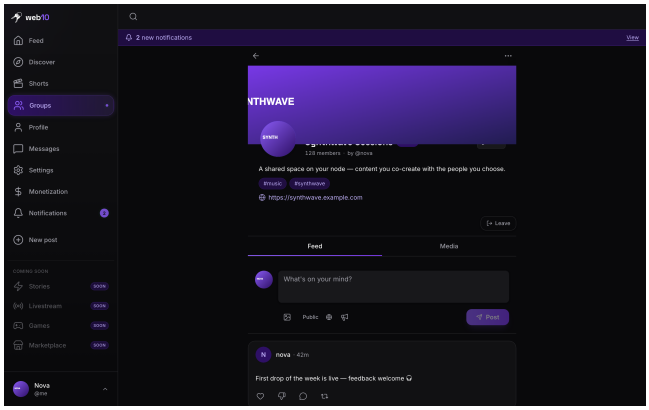


Figure 2: A group—"a shared space on your node." Membership, roles, and per-post visibility (Public / node / group) expose application choices over the generic groups and grants of §2.

Table 3: Data producers and consumers in the connected demonstration. Other social services are omitted.

Application	Writes	Reuses
web10-social	`profile`	—
Snake	`snake_scores`	`profile`
Marketplace	`marketplace_listings`	`profile`

```
FROM snake_scores s LEFT JOIN latest_profiles p
  ON s.author_key = p.author_key
ORDER BY score DESC LIMIT 50
```

Application permission to read scores does not confer permission to read profiles; an app-level profile grant does not itself grant access to a private profile group. This outer join can retain a score without a readable profile. The independent group control in §6.1 checks the corresponding inner-join exclusion while the score records remain readable. Applications therefore compose authorized relations, not unrestricted copies of another application's data.

Figure 3 shows the resulting player and seller identities. The synthetic name was edited through the social UI, then read by both applications; neither demo writes or recreates the profile. The connected checks below verify the underlying records as well as their rendered names.

5.5 Durability and extensibility

Application-named services let different clients interpret documents without changing the shared document schema. A conversation can be a group with message documents; a music library can define track services and listener roles; a shop can define catalog services. Domain-specific transactions, payment workflows, search indexes, schema evolution, and external integrations can extend this common substrate.

The durable separation is between stored content/access relationships and a particular frontend. Replacing a frontend can reuse those primitives when it understands their semantics. The user-addressed network model of §3 is the architectural answer to provider migration: the user's identity and grants are globally addressable, so moving providers is a matter of the cross-node read and membership path completing, not a redesign. That path is the active build (§6.5).

6 EVALUATION

The evaluation has four parts: a connected interoperability demonstration with real browser consent and revocation, an isolated two-node correctness scenario, a measurement of metadata fan-out cost, and a deployed operating point. The first three use synthetic data on isolated test stacks; the operating point is a read-only observation of the production node. None is a capacity or scale result.

6.1 Connected interoperability

Browser reuse. A fresh account logged into web10-social through its authenticator popup and edited its profile name to Alex Morgan. The browser approved Snake's app request, earned and saved ten points, and displayed the joined profile. After app-contract revocation, a new query returned 403 and a leaderboard refresh cleared the result and displayed an access error. After sign-out, Marketplace requested its service permission through the popup, published a listing, and displayed the same profile. Reloading the listing's detail URL preserved that result. API assertions checked exact authors, row counts, profile bodies and listing identity, so the username fallback alone could not satisfy the test.

Independent access controls. A separate HTTP case used three accounts and two social-written profiles. It checked exact two-author score/profile and listing/profile joins. With no app permission for `profile`, both queries were denied; granting it enabled the joins, and revoking the app contract denied them again. A separately labeled private-profile fixture isolated the group gate: membership enabled an exact inner join; removal emptied it while the scores remained readable. The fixture did not replace the social-written profiles in the reuse checks. HTTP contracts were installed through the API; the browser sequence above exercised the actual consent UI.

All three cases—desktop at 1440 pixels, mobile at 375 pixels, and HTTP correctness—passed in the final fresh-stack run on 2 November 2026, with zero retries or skips. Six flow captures and two identity-card captures per viewport were retained and inspected. Iteration exposed a Marketplace race: the initial listing read could overwrite a successful publication's status. Keeping publication disabled until that read settled repaired it; a focused regression failed before the repair and passed afterward.

Provenance and scope. The run used revision `db897bccb` plus a retained patch/source overlay on a four-vCPU, 8-GB DigitalOcean sfo2 host, with ClickHouse 24.8.14.39, Playwright 1.63.0 and Chromium 153. The artifact contains source/config hashes, image identities, results and all 16 original captures. The API, authenticator and social frontend used development configurations; the marketing frontend was built. Schema initialization preceded API startup, avoiding the inherited fresh-stack ordering failure. Project storage and the droplet were removed after collection.

The hosted apps share one browser origin, although the HTTP case supplies a separate app Origin. They demonstrate reuse across applications, not separate browser-origin isolation or an independently recruited developer study. Clients know the profile format; automatic schema discovery is outside this result. Initial-letter avatars are not media-delivery evidence. These controls establish named grant/revocation behavior, not a stale-access or capability-expiry bound.

6.2 Cross-node identity and local revocation

We ran a synthetic correctness scenario across two API processes, separate ClickHouse and MinIO instances, independently generated HS256 signing secrets, and real HTTPS home-issuer certification with an ephemeral test CA. Bob creates a private group and document through B's API. Alice's A-issued JWT initially fails both B's document read and query; giving a distinct B-side Alice the membership also fails to authorize A's Alice. Bob then grants the exact foreign principal through the owner API. Both read paths return the exact document; removing that membership denies both, and regrant restores both.

All steps reuse the same JWT. A's TLS proxy records exactly one successful certification across this lifecycle: B rechecks access while retaining cached identity. A foreign-provider JWT signed with B's key is rejected twice, with one failed issuer call and a negatively cached second rejection. An independent Origin-bearing query gate denies a local namesake's app grant, allows a qualified foreign app-contract fixture, and denies again after its revocation; direct group-authorized reads remain available.

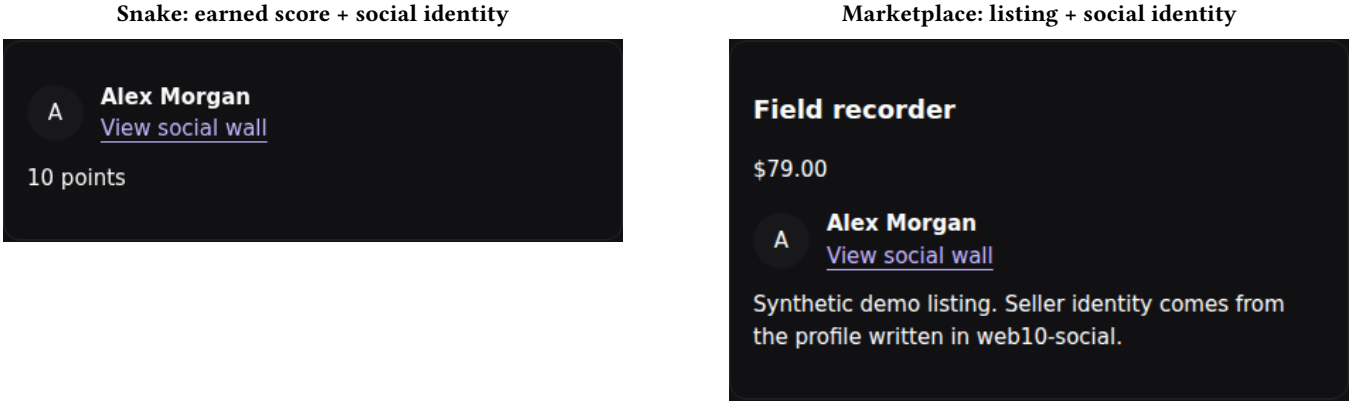


Figure 3: One social-written profile, two application views. Snake joins an earned score to the player's profile; Marketplace joins a listing to the seller's same profile. These are direct browser element captures, with synthetic account and listing data. Full-page, consent, and denial captures are retained in the artifact.

Three isolated correctness runs completed, not deployment or load trials. An initial attempt failed on a harness import path before scenario execution. The first pass used revision `04da544442c3` plus the retained owner-list repair patch; a second pass used committed revision `3be1f1ec10fd` with an empty source patch. The latest replays clean merged dev revision `0acc771e2efc` after PR #1360 merged, again with an empty patch and no overlay. All ran without scenario retries. Docker-internal, globally classified addresses exercise the unchanged public-address guard; the test CA and network do not establish public-internet interoperability. Foreign app grant/revocation use labelled service-function fixtures, not browser consent or completed foreign enrollment. Owner grant/removal is not foreign self-join. Receipts, executable source, image identities and verified cleanup are retained in the experiment artifact.

6.3 The metadata fan-out cost of the access layer

The access layer resolves the reader's readable groups before the query executes. We measured that cost to understand the design, not to report a latency distribution: the question was whether per-group metadata resolution scales with the number of candidate groups, and what a batched resolution buys. Table 4 reports one sequential trial on a DigitalOcean sfo2 `s-4vcpu-8gb` host (four vCPUs, 8 GB nominal RAM, Ubuntu 24.04, Linux 6.8, Docker 29.1.3), co-locating a single Uvicorn process and ClickHouse 24.3.18.7. The synthetic seed created 1,000 creators with one small post each and a reader who joined all their groups; those groups also granted public reads. Personal queries supplied $K = 10$, 100, or 1000 candidate groups on this same dataset and host. A separate discover query supplied one board group ($K = 1$), containing ten additional posts. Queries selected document IDs and bodies, ordered by creation time and ID, with a limit of 20. They did not execute engagement joins, media preparation or browser page assembly. HTTP requests supplied no Origin header, so this trial does not exercise the browser-origin app-contract gate.

Variants. The per-group resolver performs point metadata queries for each candidate group. The batched resolver fetches memberships, principal-class grants and role definitions across the candidate set in three database queries, then resolves access in memory. This reduces round trips, not the amount of permission metadata to process. We measured each resolver with real document execution (B) and a fixed 20-row synthetic scan result (C). C retains the policy/compilation path but is not an output-equivalent query execution. A third variant (A) precompiles the known-readable projection and times ClickHouse execution directly, outside the HTTP handler.

Finding. The per-group resolver's cost grows linearly with the number of candidate groups: increasing K from 10 to 1,000 increased median readable time by 100.44×. The batched resolver, which fetches memberships, principal-class grants, and role definitions in three queries and resolves in memory, grows 9.12× over the same range—and at $K = 1000$ reduces median readable time by 35.22× and median handler time by 21.04× relative to per-group. The per-group path still completes requests, but its 7.64-second median at $K = 1000$ is a practical bottleneck for a reader who follows many creators; the batched path's 363 ms is not. This is the design decision the measurement drives: the query endpoint should use the batched resolver for large candidate sets.

The tradeoff is honest in both directions. Batching is not uniformly cheaper: at the discover board's $K = 1$, the per-group path is faster (9.53 ms readable versus 25.37 ms batched), because three fixed round trips dominate when there is one group to check. And batching does not make the cost constant: the batched handler median still grew 4.47× over the 100-fold increase in K, as the metadata to process and the compiled query to emit both grow with the candidate set. The result is a characterization of the access layer's cost structure— metadata fan-out is the dominant term at large K, and batching is the mitigation—not a claim of constant latency or a performance benchmark.

The trial is a single sequential run (20 samples per cell, 400 measurements, one host, localhost HTTP, no concurrency, no warm-up exclusion), and the query is a simplified projection

Table 4: Personal-query results from one sequential trial on the same 1,000-creator dataset, 20 samples per cell. B executes the document query; C substitutes a fixed 20-row result. Times are milliseconds. Readable denotes permission-resolution wall time; total denotes internal handler time, not end-to-end HTTP latency.

K	Resolver	Variant	Readable median	Readable p95	Total median	Total p95
10	Per-group	B	75.03	93.30	129.36	171.18
10	Batched	B	23.47	31.17	81.28	98.91
100	Per-group	B	764.39	843.21	831.35	915.18
100	Batched	B	41.78	51.65	99.42	139.47
1000	Per-group	B	7535.71	8504.61	7644.66	8679.24
1000	Batched	B	213.98	284.03	363.31	435.03
10	Per-group	C	76.36	88.84	104.96	134.30
10	Batched	C	27.07	33.60	59.91	85.53
100	Per-group	C	693.68	806.76	731.74	848.74
100	Batched	C	40.45	49.45	71.79	90.97
1000	Per-group	C	8388.39	9102.21	8529.75	9204.19
1000	Batched	C	222.57	281.20	299.59	350.27

without engagement joins, media preparation, or browser page assembly. It measures the access layer's cost structure on a representative workload; it does not establish a latency distribution, a capacity bound, or a causal authorization-overhead estimate. The direct-scan baseline (A) excludes authentication, compilation, and serialization, so $B - A$ is a pipeline-versus-scan comparison, not a matched causal measurement. Provenance: source revision `1a983520e`, seed 42, ClickHouse 24.3.18.7, single Uvicorn process; an earlier 100-creator trial on a different host and snapshot showed the same qualitative tradeoff and is not pooled with this table.

6.4 Deployed operating point

The platform is deployed and serving authenticated users. A read-only observation of the production node on 28 October 2026 recorded 597 account rows, 593 live documents across 30 distinct authors and 10 services, 49 group contracts, and 1,298 membership rows, on a host running the social, authenticator, and API frontends. Nine authenticated browser navigations (following feed, music, groups) returned HTTP 200 with a retained session and zero uncaught page errors; the following feed rendered 20 posts in a median 5,396 ms of full-page readiness. This is an operating point—evidence the system runs and serves real users—not a capacity or scale result: the host also runs a development stack, the observation is a snapshot without an activity window, and the readiness figures include frontend startup and network work, not server-query duration.

6.5 Future work

The evaluation above is bounded by construction, and the open work is specific.

Federation beyond the correctness slice. The isolated experiment in §6.2 exercises home-issuer certification and owner-managed foreign grant/removal, not a complete federated product. Independent production signing-key provisioning, complete local-principal migration, foreign self-join and app enrollment, multiworker/restart behavior, public deployment and client-side feed merging remain open. Cached identity answers intentionally last until JWT expiry; this does not establish immediate issuer-side revocation or revoke already-issued media capabilities. JWKS would be an optional verification optimization, not a prerequisite for the live-call model.

Workload and baseline. The metadata fan-out measurement is a single sequential trial on a simplified query. Full-feed latency (including engagement joins, media preparation, and browser page assembly), concurrent capacity, mixed allowed/denied access, and a matched shared-table policy baseline (PostgreSQL with row-level security, same data and access semantics) remain to be measured. A causal authorization-overhead estimate requires an isolated ablation with equivalent preauthorized result sets, not the pipeline-versus- scan comparison reported here.

Correctness. Security evidence consists of regression tests and the named connected controls. Whole-system conformance, revocation-latency bounds (racing grant changes with reads through metadata visibility, in-flight requests, and issued capabilities), and the media capability boundary remain open. Direct read-URL refresh enforces the verified caller's exact owner namespace; that check alone does not establish stored-reference provenance or revoke previously issued capabilities. Parser/engine semantic differences and result-projection provenance require targeted testing.

7 RELATED WORK

The contribution is a *combination*, not a single invention. Each capability in Table 1 has prior art; the gap is that no prior system combines all five—user-owned data in any format, user-managed fine-grained access, access that is cross-node-addressable, composable application-defined query, and a user-addressed network—in one system. We position web10 against the closest work in each dimension and state the specific gap each leaves.

Decentralized social data. ActivityPub [11] (Mastodon) defines client/server and federated server/server activity delivery; its authorization is platform-defined—follow/block and collection-level visibility—not per-principal grants over application-defined services. ATProto [10] (Bluesky) separates repositories, relays, and AppViews, supporting user agency and provider switching;

its visibility model is likewise collection-level, with AppViews providing application-defined *reads* but not user-managed *grants*. Both decentralize the data plane; neither decentralizes the access layer. web10's distinction is directional: the access model is user-managed and service-scoped, and the query is application-defined, not platform-defined.

Solid: the closest prior work. Solid [15] separates applications from personal data stores (Pods), with social application work [14] and a maintained Community Solid Server [7]. Its access model is the most developed of any decentralized social system: WAC supports resource permissions and group agents through `acl:agentGroup` [5], and ACP adds agent, client, and issuer matching with allow/deny mode sets [3]. web10 does not claim Solid lacks group or client authorization; it has both. The distinction is in the *shape* of the model. Solid's access control is *resource-centric* —ACLs attached to linked-data resources and containers, evaluated per resource. web10's is *service-centric* —a uniform role/grant vocabulary (principal class × service × operation) in which a role is a per-service permission map, the same shape across the app-trust and person-trust layers, and the grant is a row naming a globally addressable principal. Concretely: Solid answers "who may read or write this *resource*?"; web10 answers "what *operations* may this *principal class* perform on this *service*, in this *group*, under this *app contract*?" The latter is the IAM policy shape (principal, service, operation, grant) exposed to the end user, not an infrastructure operator. A second distinction is the query surface: Solid's cross-Pod query cost depends on deployment, caching, and linked-data traversal strategy; web10 composes application-defined SQL over a shared columnar store with the access decision made on the input (§4). A performance comparison would require a matched workload and deployment; we do not claim one here.

Cloud IAM and relationship-based authorization. AWS IAM (documentation) [1] and the RBAC/ABAC literature [6, 8, 16] provide the policy concepts web10 adapts at the user boundary: principals, roles, service-scoped operations, revocable grants. The move is directional—a familiar policy vocabulary exposed to *users* about their own content and groups, orchestrated by the API and query layer, rather than to infrastructure operators about cloud resources. At industrial scale, Zanzibar [12] is the closest prior art in *relationship-based* authorization: a uniform ACL data model and configuration language across hundreds of services, with consistency as a first-class problem (causal ordering of ACL updates, snapshot reads). Zanzibar's model is *infrastructure-side*—the operator defines the relationships and the service enforces them; the end user is not the policy administrator. web10 inverts the boundary: the *user* defines the grants, the *principal is globally addressable across nodes*, and the enforcement is at the query layer over a shared columnar store. Zanzibar's consistency guarantees motivate the explicit revocation-visibility evaluation we leave as forward scope (§6.5); we do not claim equivalent consistency here. TAO [4] provides the deployed precedent for *read-time* privacy checking at social-graph scale (a billion reads/sec, privacy check per item, pull not push); web10's query boundary is the same idea—every row access-checked on its way out—applied to service-namespaced documents in a columnar store with the check expressed as a reader-filtered CTE rather than a per-item predicate.

Multi-tenant isolation and row-level security. The database literature on tenant isolation—shared tables with an organization identifier (Force.com [18]), schema-mapping techniques [2], and the shared-versus-separate spectrum [9]—is the closest systems ancestor. web10's key-space per user makes the *end user* the logical tenant, taking the spectrum to its limit: not one tenant per organization, but one logical tenant per end user, in a shared columnar store. Authorship uses `author_key`; shared reads use authorized group relations rather than a physical database boundary. PostgreSQL row-security policies [13] are the alternative enforcement location: per-row policy evaluation inside the database engine. web10 places enforcement above the engine—the reader-filtered CTE is constructed in the application layer before the query reaches ClickHouse [17]—which is what makes the access decision composable with application-defined joins and aggregation. A matched policy and workload comparison (§6) can distinguish the costs of these placements; we report the design difference here and leave the cost comparison as forward scope.

8 CONCLUSION

web10 extends the email model—global identifiers, sovereign stores, client-mediated communication, no central authority—from one data format to all formats, and from content routing to access control. Content and access relationships live in a shared columnar data plane; the query layer supplies the reader's service relations before joins and aggregation; and the access model is globally addressable, so a user's grants reach across the nodes. The deployed social application, and the game and marketplace that reuse its profiles, show the separation working: ranked feeds and scoped engagement reads with query definitions in the application, and cross-app composition without a node change.

The connected game and marketplace extend that separation beyond the social interface: each joins its own records to social-written profiles without a new node endpoint or a change to the social application. Browser consent and cross-author app/group controls make this a demonstrated composition path, rather than a hypothetical extension to other application domains.

The metadata fan-out measurement identifies per-group resolution as the access layer's dominant cost at scale: at 1,000 candidate groups, batching reduced median handler time from 7.64 s to 363 ms, while costing more at one group. This is a design decision, not a scalability claim—and it is the kind of concrete, measured tradeoff that an operating system earns. The forward scope is specific: federation beyond the isolated correctness slice (§6.2), full-feed and concurrent workload measurements, a matched shared-table policy baseline, and revocation-latency bounds. The architectural contribution is the integration itself: a user-addressed network in which applications compose social views over reader-specific service relations in a shared columnar store, and the user's access policy reaches across the nodes.

REFERENCES

[1] Amazon Web Services. 2025. AWS Identity and Access Management (IAM) User Guide. Amazon Web Services documentation. https://docs.aws.amazon.com/IAM/latest/UserGuide/introduction.html

[2] Stefan Aulbach, Torsten Grust, Dean Jacobs, Alfons Kemper, and Jan Rittinger. 2008. Multi-Tenant Databases for Software as a Service: Schema-Mapping Techniques. In *Proceedings of the 2008 ACM SIGMOD International Conference on Management of Data (SIGMOD '08)*. ACM, 1195–1206.

[3] Matthieu Bosquet. 2022. Access Control Policy (ACP). W3C Solid Community Group, Editor's Draft. https://solid.github.io/authorization-panel/acp-specification/

[4] Nathan Bronson, Zach Amsden, George Cabrera, Prasad Chakka, Peter Dimov, Hui Ding, Jack Ferris, Anthony Giardullo, Sachin Kulkarni, Harry C. Li, Mark Marchukov, Dmitri Petrov, Lovro Puzar, Yee Jiun Song, and Venkateshwaran Venkataramani. 2013. TAO: Facebook's Distributed Data Store for the Social Graph. In *Proceedings of the 2013 USENIX Annual Technical Conference (USENIX ATC '13)*. USENIX Association, 49–60.

[5] Sarven Capadisli. 2022. Web Access Control. W3C Solid Community Group, Version 1.0.0-cr-1. https://solidproject.org/TR/wac

[6] David F. Ferraiolo and David R. Kuhn. 1992. Role-Based Access Controls. In *Proceedings of the 15th National Computer Security Conference*. 554–563.

[7] Joachim Van Herwegen and Ruben Verborgh. 2024. The Community Solid Server: Supporting Research and Development in an Evolving Ecosystem. *Semantic Web* 15, 6 (2024), 2597–2611. https://doi.org/10.3233/SW-243726

[8] Vincent C. Hu, David F. Ferraiolo, David R. Kuhn, Adam Schnitzer, Kenneth Sandlin, Robert Miller, and Karen Scarfone. 2014. *Guide to Attribute Based Access Control (ABAC) Definition and Considerations*. Technical Report SP 800-162. National Institute of Standards and Technology. Includes updates as of August 2019.

[9] Dean Jacobs and Stefan Aulbach. 2007. Ruminations on Multi-Tenant Databases. In *Proceedings of BTW 2007*. 514–521.

[10] Martin Kleppmann, Paul Frazee, Jake Gold, Jay Graber, Daniel Holmgren, Devin Ivy, Jeromy Johnson, Bryan Newbold, and Jaz Volpert. 2024. Bluesky and the AT Protocol: Usable Decentralized Social Media. In *Proceedings of the ACM CoNEXT-2024 Workshop on the Decentralization of the Internet (DIN '24)*. ACM, 9. https://doi.org/10.1145/3694809.3700740 Extended version: arXiv:2402.03239.

[11] Christine Lemmer-Webber, Jessica Tallon, Erin Shepherd, Amy Guy, and Evan Prodromou. 2018. ActivityPub. W3C Recommendation. https://www.w3.org/TR/activitypub/ Editors: Christine Lemmer-Webber and Jessica Tallon.

[12] Ruoming Pang, Ramon Caceres, Mike Burrows, Zhifeng Chen, Pratik Dave, Nathan Germer, Alexander Golynski, Kevin Graney, Nina Kang, Lea Kissner, Jeffrey L. Korn, Abhishek Parmar, Christina D. Richards, and Mengzhi Wang. 2019. Zanzibar: Google's Consistent, Global Authorization System. In *Proceedings of the 2019 USENIX Annual Technical Conference (USENIX ATC '19)*. USENIX Association. https://www.usenix.org/system/files/atc19-pang.pdf

[13] PostgreSQL Global Development Group. n.d. PostgreSQL Documentation: Row Security Policies. PostgreSQL documentation. https://www.postgresql.org/docs/current/ddl-rowsecurity.html Accessed October 2026.

[14] Andrei Sambra, Amy Guy, Sarven Capadisli, and Nicola Greco. 2016. Building Decentralized Applications for the Social Web. In *Proceedings of the 25th International Conference Companion on World Wide Web (WWW '16 Companion)*. ACM, 1033–1034.

[15] Andrei Vlad Sambra, Essam Mansour, Sandro Hawke, Maged Zereba, Nicola Greco, Abdurrahman Ghanem, Dmitri Zagidulin, Ashraf Aboulnaga, and Tim Berners-Lee. 2016. *Solid: A Platform for Decentralized Social Applications Based on Linked Data*. Technical Report. MIT CSAIL and Qatar Computing Research Institute. http://emansour.com/research/lusail/solid_protocols.pdf

[16] Ravi S. Sandhu, Edward J. Coyne, Hal L. Feinstein, and Charles E. Youman. 1996. Role-Based Access Control Models. *IEEE Computer* 29, 2 (1996), 38–47.

[17] Robert Schulze, Tom Schreiber, Ilya Yatsishin, Ryadh Dahimene, and Alexey Milovidov. 2024. ClickHouse: Lightning Fast Analytics for Everyone. *Proceedings of the VLDB Endowment* 17, 12 (2024), 3731–3744. https://doi.org/10.14778/3685800.3685802

[18] Craig D. Weissman and Steve Bobrowski. 2009. The Design of the Force.com Multitenant Internet Application Development Platform. In *Proceedings of the 2009 ACM SIGMOD International Conference on Management of Data (SIGMOD '09)*. ACM, 889–896. https://doi.org/10.1145/1559845.1559942